

Gudy Examination Series

MAP YOUR MASTERY. MASTER YOUR GOALS.

Sertifikasi Kompetensi (BNSP) JWD & LKS RPL Laravel

Web Security Hardening (OWASP Top 10)

Proteksi Serangan CSRF, XSS Auto-escaping, SQL Injection Parameterized Queries, Mass Assignment, Gates & Policies

Kode Paket:	GDY-SER-WEB_-100
Jumlah Soal:	20 Butir Pilihan Ganda
Alokasi Waktu:	60 Menit
Target Nilai:	100 / Sempurna

Petunjuk Pengerjaan Ujian Mandiri:

- Kerjakan seluruh butir soal secara mandiri tanpa membuka buku catatan atau bantuan mesin pencari untuk mengukur pemahaman murni Anda.
- Pilihlah satu jawaban yang paling tepat (A, B, C, D, atau E) untuk setiap butir soal.
- Perhatikan alokasi waktu ujian (60 menit) untuk melatih kecepatan membaca dan ketepatan kalkulasi.
- Setelah selesai, periksalah lembar jawaban Anda dengan *Kunci Jawaban & Lembar Pembahasan Lengkap* yang terletak di halaman akhir dokumen ini.

BAGIAN 1: NASKAH SOAL PILIHAN GANDA

Soal #1

Cross-Site Request Forgery (CSRF)

Bagaimana framework Laravel secara otomatis memproteksi aplikasi web dari serangan pemalsuan permintaan lintas situs (CSRF)?

- A. Dengan memblokir semua form input
- B. Melalui middleware `VerifyCsrfToken` yang memvalidasi token rahasia per-sesi pada setiap request POST/PUT/DELETE via direktif `@csrf`
- C. Dengan mematikan koneksi internet klien
- D. Dengan mewajibkan pengguna login setiap 5 menit
- E. Melalui enkripsi gambar

Soal #2

Cross-Site Scripting (XSS)

Bahaya penggunaan sintaks Blade `{!! \$content !!}` tanpa pembersihan (*unfiltered raw output*) adalah membuka celah...

- A. SQL Injection
- B. Stored / Reflected Cross-Site Scripting (XSS)
- C. Denial of Service (DoS)
- D. DNS Spoofing
- E. Man in the Middle

Soal #3

SQL Injection Prevention

Manakah di antara potongan kode query berikut yang RENTAN terhadap serangan SQL Injection?

- A. `User::where('email', $request->email)->first();`
- B. `DB::table('users')->where('id', $id)->get();`
- C. `DB::select("SELECT * FROM users WHERE email = '" . $request->email . "'");`
- D. `DB::select("SELECT * FROM users WHERE email = ?", [$request->email]);`
- E. `User::find($id);`

Soal #4

Mass Assignment Vulnerability

Celah keamanan Mass Assignment terjadi ketika penyerang memanipulasi form request untuk mengubah kolom sensitif (seperti `is\_admin = true`). Fitur Eloquent yang mencegah celah ini adalah...

- A. Mendeklarasikan array properti `fillable` secara eksplisit pada model
- B. Mengatur `guarded = []`
- C. Menonaktifkan database
- D. Menghapus file migration
- E. Menggunakan PHP 7

Soal #5

Hashing Password Aman

Fungsi bawaan Laravel yang digunakan untuk mengenkripsi kata sandi menggunakan algoritma hashing satu arah yang aman (Bcrypt / Argon2id) adalah...

- A. `md5($password);`
- B. `sha1($password);`
- C. `Hash::make($password);`
- D. `base64_encode($password);`
- E. `crypt_string($password);`

Soal #6

Otorisasi via Laravel Policy

Perintah artisan untuk membuat class Policy baru yang terikat pada model `Post` adalah...

- A. `php artisan make:policy PostPolicy --model=Post`
- B. `php artisan make:guard PostPolicy`
- C. `php artisan create:policy PostPolicy`
- D. `php artisan make:rule PostPolicy`
- E. `php artisan auth:policy PostPolicy`

Soal #7

Pengecekan Otorisasi di Controller

Di dalam controller, cara mengeksekusi otorisasi agar hanya pemilik artikel yang boleh mengedit postingannya adalah...

- A. `$this->authorize('update', $post);`
- B. `if (Auth::user()->id != $post->user_id) { abort(403); }`
- C. `Gate::check('update');`
- D. Jawaban A dan B benar
- E. Hanya B yang benar

Soal #8

Direktif Blade @can

Untuk menyembunyikan tombol 'Hapus Post' di tampilan Blade jika user tidak memiliki izin otorisasi, direktif yang digunakan adalah...

- A. @can('delete', \$post) ... @endcan
- B. @auth ... @endauth
- C. @admin ... @endadmin
- D. @protect ... @endprotect
- E. @check ... @endcheck

Soal #9

Brute Force Attack Mitigation

Mekanisme pengamanan yang diterapkan Laravel Fortify / Breeze pada formulir login untuk mencegah serangan tebak password massal (*brute force*) adalah...

- A. Menghapus akun user setelah 1 kali salah password
- B. Rate limiting dengan penguncian sementara akun (*lockout throttle*) setelah 5 percobaan gagal berturut-turut
- C. Mengirim SMS berbayar ke pengguna
- D. Menutup website pada malam hari
- E. Mengganti password secara acak

Soal #10

File Upload Security (MIME Spoofing)

Mengapa sekadar memeriksa ekstensi file (.jpg atau .png) TIDAK CUKUP aman saat menangani upload file?

- A. Karena ekstensi file dapat dipalsukan oleh penyerang dengan menamai file skrip berbahaya menjadi `shell.php.jpg`
- B. Karena gambar jpg tidak bisa ditampilkan di Chrome
- C. Karena ekstensi file memakan memori server
- D. Karena PHP tidak bisa membaca ekstensi
- E. Karena Windows tidak mendukung ekstensi

Soal #11

Strict Session Hijacking Prevention

Untuk mencegah pembajakan sesi (*session hijacking*) melalui jaringan yang tidak aman, cookie sesi di file `config/session.php` harus diatur dengan nilai...

- A. 'secure' => true, dan 'http_only' => true
- B. 'secure' => false
- C. 'http_only' => false
- D. 'encrypt' => false
- E. 'lifetime' => 999999

Soal #12

Celah Insecure Direct Object References (IDOR)

Celah keamanan di mana pengguna dapat melihat atau mengedit faktur transaksi pengguna lain hanya dengan mengganti ID pada URL (`/invoices/105` menjadi `/invoices/106`) disebut...

- A. XSS
- B. IDOR (Insecure Direct Object References)
- C. SQL Injection
- D. Phishing
- E. Buffer Overflow

Soal #13

Enkripsi Simetris APP_KEY

Parameter `APP\_KEY` di file `.env` Laravel digunakan untuk...

- A. Kata sandi login admin utama
- B. Kunci kriptografi 32 karakter untuk mengenkripsi cookie sesi, payload session, dan enkripsi simetris data sensitif
- C. Lisensi berbayar ke perusahaan Laravel
- D. Nomor seri prosesor komputer server
- E. Nama database MySQL

Soal #14

Bahaya Mode Debug di Server Produksi

Mengapa pengaturan `APP\_DEBUG=true` sangat dilarang dibiarkan aktif di server produksi?

- A. Karena aplikasi tidak bisa dibuka pengguna
- B. Karena saat terjadi error, sistem akan menampilkan stack trace lengkap yang membeberkan kredensial database, path file server, dan environment variables ke publik
- C. Karena tampilan web menjadi hitam putih
- D. Karena PHP akan otomatis terhapus
- E. Karena website tidak bisa diakses lewat handphone

Soal #15

Content Security Policy (CSP)

Header keamanan HTTP `Content-Security-Policy` (CSP) berfungsi untuk...

- A. Mengatur jam buka tutup website
- B. Membatasi domain mana saja yang diizinkan memuat skrip, gaya, gambar, dan frame pada halaman web untuk menekan risiko XSS
- C. Menghapus cache browser otomatis
- D. Mempercepat loading font
- E. Memblokir iklan televisi

Soal #16

Gate vs Policy

Dalam sistem otorisasi Laravel, perbedaan fungsi utama antara Gate dan Policy adalah...

- A. Gate biasanya untuk aksi umum non-model (misal: `access-admin-panel`), sedangkan Policy dikelompokkan secara khusus di sekitar Model tertentu (misal: `PostPolicy`)
- B. Gate hanya berjalan di PHP 7
- C. Policy tidak memerlukan class PHP
- D. Gate hanya untuk menghapus user
- E. Tidak ada perbedaan fungsi

Soal #17

Validasi Email DNS

Aturan validasi `email` => `required|email:rfc,dns` di Laravel melakukan pengecekan...

- A. Format sintaks email sesuai standar RFC DAN memeriksa keberadaan record MX DNS domain email tersebut secara online
- B. Mengirim email verifikasi otomatis ke kotak masuk
- C. Memaksa email harus berakhiran @gmail.com
- D. Menghubungkan email ke akun media sosial
- E. Memeriksa kata sandi email

Soal #18

Sanitasi Input HTML Purifier

Jika sebuah aplikasi blog mewajibkan input teks berformat kaya (WYSIWYG/Rich Text Editor), cara paling aman menangani konten HTML dari user adalah...

- A. Menyimpan dan menampilkannya langsung dengan `{!! \$html !!}` tanpa filter
- B. Memfilter konten menggunakan library pembersih HTML seperti HTMLPurifier untuk membersihkan tag berbahaya (*script, onerror, iframe*)
- C. Menolak semua input yang memuat huruf vokal
- D. Menyimpan data sebagai file teks biasa tanpa format
- E. Mengubah semua huruf menjadi huruf kecil

Soal #19

Regenerasi ID Sesi Login

Mengapa method `Auth::attempt()` di Laravel secara otomatis meregenerasi session ID (`\$request->session()->regenerate();`) saat login berhasil?

- A. Untuk mencegah serangan *Session Fixation*
- B. Untuk mempercepat transfer data
- C. Agar memori browser bersih
- D. Untuk menghapus database
- E. Agar password tidak terlihat

Soal #20

Prinsip Least Privilege (Hak Istimewa Terkecil)

Prinsip keamanan *Least Privilege* dalam pengelolaan akun basis data aplikasi menyatakan bahwa...

- A. Aplikasi web harus selalu menggunakan akun `root` MySQL agar tidak pernah kekurangan hak akses
- B. Pengguna aplikasi web hanya boleh diberikan hak akses seminimal mungkin yang diperlukan untuk menjalankan fungsinya (misal: hanya SELECT, INSERT, UPDATE, DELETE; bukan DROP DATABASE atau GRANT)
- C. Setiap pengguna berhak menjadi administrator
- D. Password database boleh dibagikan ke seluruh karyawan
- E. Database tidak perlu diberi password

BAGIAN 2: KUNCI JAWABAN & PEMBAHASAN LENGKAP

No	Topik / Pokok Bahasan	Kunci	No	Topik / Pokok Bahasan	Kunci
1	Cross-Site Request Forgery (CSRF)	B	11	Strict Session Hijacking Prevention	A
2	Cross-Site Scripting (XSS)	B	12	Celah Insecure Direct Object References (IDOR)	B
3	SQL Injection Prevention	C	13	Enkripsi Simetris APP_KEY	B
4	Mass Assignment Vulnerability	A	14	Bahaya Mode Debug di Server Produksi	B
5	Hashing Password Aman	C	15	Content Security Policy (CSP)	B
6	Otorisasi via Laravel Policy	A	16	Gate vs Policy	A
7	Pengecekan Otorisasi di Controller	D	17	Validasi Email DNS	A
8	Direktif Blade @can	A	18	Sanitasi Input HTML Purifier	B
9	Brute Force Attack Mitigation	B	19	Regenerasi ID Sesi Login	A
10	File Upload Security (MIME Spoofing)	A	20	Prinsip Least Privilege (Hak Istimewa Terkecil)	B

Lembar Pembahasan Langkah per Langkah:

Pembahasan Soal #1 [Cross-Site Request Forgery (CSRF)] — Kunci Jawaban: (B)

Setiap form HTML wajib memuat token CSRF acak yang cocok dengan token sesi server. Penyerang dari website pihak ketiga tidak dapat membaca atau menebak token ini.

Pembahasan Soal #2 [Cross-Site Scripting (XSS)] — Kunci Jawaban: (B)

`{!! $content !!}` merender HTML mentah tanpa fungsi `htmlspecialchars()`. Jika konten memuat tag jahat seperti `<script>stealCookie()</script>`, skrip tersebut akan dieksekusi langsung di browser korban.

Pembahasan Soal #3 [SQL Injection Prevention] — Kunci Jawaban: (C)

Potongan C menggabungkan (*concatenation*) string input mentah pengguna langsung ke query SQL tanpa parameter binding, memungkinkan penyerang menyisipkan payload seperti `` OR '1'='1``.

Pembahasan Soal #4 [Mass Assignment Vulnerability] — Kunci Jawaban: (A)

Properti `$fillable` mendefinisikan whitelist atribut yang boleh diisi secara massal via `create()` atau `update()`. Atribut di luar daftar ini otomatis diabaikan.

Pembahasan Soal #5 [Hashing Password Aman] — Kunci Jawaban: (C)

`Hash::make()` menggunakan Bcrypt/Argon2id dengan salting acak dan *work factor* yang dapat diatur, menjamin ketahanan terhadap serangan kamus dan rainbow tables (tidak seperti MD5/SHA1 yang usang).

Pembahasan Soal #6 [Otorisasi via Laravel Policy] — Kunci Jawaban: (A)

``make:policy PostPolicy --model=Post`` menghasilkan class policy dengan boilerplate method lengkap (``view``, ``create``, ``update``, ``delete``).

Pembahasan Soal #7 [Pengecekan Otorisasi di Controller] — Kunci Jawaban: (D)

Keduanya benar, namun ``$this->authorize('update', $post)`` adalah *best practice* berbasis Policy yang melempar exception 403 secara otomatis.

Pembahasan Soal #8 [Direktif Blade @can] — Kunci Jawaban: (A)

``@can('ability', $model)`` mengevaluasi policy secara deklaratif di template Blade.

Pembahasan Soal #9 [Brute Force Attack Mitigation] — Kunci Jawaban: (B)

Throttle limiter mengunci akses login dari IP/email tertentu selama 60 detik jika mendeteksi kegagalan berulang.

Pembahasan Soal #10 [File Upload Security (MIME Spoofing)] — Kunci Jawaban: (A)

Penyerang dapat mengganti nama skrip berbahaya (webshell) dengan ekstensi ``.jpg``. Validasi yang benar harus memeriksa MIME type biner asli menggunakan rule ``image`` atau pustaka `FileInfo`.

Pembahasan Soal #11 [Strict Session Hijacking Prevention] — Kunci Jawaban: (A)

``secure => true`` memastikan cookie hanya ditransmisikan via protokol HTTPS terenkripsi. ``http_only => true`` mencegah skrip JavaScript berbahaya membaca cookie via XSS.

Pembahasan Soal #12 [Celah Insecure Direct Object References (IDOR)] — Kunci Jawaban: (B)

IDOR terjadi ketika aplikasi menerima input ID dari pengguna dan langsung mengambil data dari database tanpa memverifikasi kepemilikan hak akses (*authorization check*).

Pembahasan Soal #13 [Enkripsi Simetris APP_KEY] — Kunci Jawaban: (B)

APP_KEY adalah kunci enkripsi AES-256-CBC yang digunakan oleh encrypter Laravel untuk mengamankan data terenkripsi dan token sesi.

Pembahasan Soal #14 [Bahaya Mode Debug di Server Produksi] — Kunci Jawaban: (B)

Halaman error debug memaparkan informasi arsitektur internal yang sangat berharga bagi hacker untuk merancang eksploitasi terarah.

Pembahasan Soal #15 [Content Security Policy (CSP)] — Kunci Jawaban: (B)

CSP adalah mekanisme pertahanan berlapis yang membatasi eksekusi resource yang tidak sah (*unauthorized scripts*).

Pembahasan Soal #16 [Gate vs Policy] — Kunci Jawaban: (A)

Gate ideal untuk izin independen berbasis peran umum, sedangkan Policy mengorganisir logika hak akses per model Eloquent.

Pembahasan Soal #17 [Validasi Email DNS] — Kunci Jawaban: (A)

Opsi `dns` memverifikasi apakah domain email yang dimasukkan benar-benar memiliki mail server aktif (MX record).

Pembahasan Soal #18 [Sanitasi Input HTML Purifier] — Kunci Jawaban: (B)

HTMLPurifier membersihkan tag dan atribut berbahaya (seperti `

Pembahasan Soal #19 [Regenerasi ID Sesi Login] — Kunci Jawaban: (A)

Session Fixation terjadi ketika penyerang memaksa korban menggunakan session ID tertentu sebelum login. Meregenerasi ID sesi baru saat login mematikan serangan ini.

Pembahasan Soal #20 [Prinsip Least Privilege (Hak Istimewa Terkecil)] — Kunci Jawaban: (B)

Least Privilege membatasi potensi dampak kerusakan jika terjadi kompromi sistem; aplikasi tidak boleh menjalankan hak administratif yang tidak dibutuhkan.